

What can your AI see?

Context, fluency, and connectors

Week 6 · Learn, Vibe, Build · October 5, 2026

Choose what it sees. Check what it produces. Control what it can change.

- 1 · Context: what the AI sees
- 2 · Fluency: how you work with it (the 4Ds)
- 3 · Connectors: what it can reach and change

Context is the working material

- It sees: instructions + chat + files + tool results
- It doesn't know your project unless you say so
- Weak: “Improve my portfolio.” Add audience, goal, constraints
- Choose what this task needs; don't dump everything

AI Fluency: the 4D framework

- Delegation · Description · Discernment · Diligence
- Effective, efficient, ethical, and safe
- Dakan (Ringling) & Feller (UCC), with Anthropic

Three ways to work with AI

- Automation: AI does a task you defined
- Augmentation: you think together
- Agency: you set AI up to act on its own later

Delegation: whether, when, how

- Know the goal and the task before the tool
- Pick the mode and the right tool
- Keep the work where the thinking is the deliverable

Description: Product · Process · Performance

- Product: what you want out (format, audience, style)
- Process: how to approach it (steps, what to read)
- Performance: how it should behave (concise? push back?)

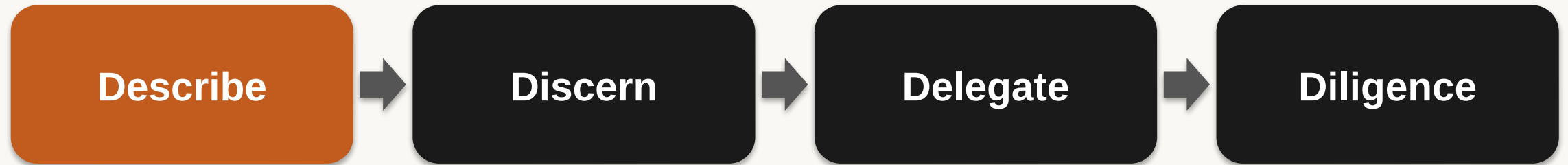
Discernment: fluent $\neq$ correct

- Judge the output and the behaviour
- Product · process · performance, again
- Check facts, run it, look closely

Diligence: responsibility doesn't transfer

- Creation: choose tools and data thoughtfully
- Transparency: be honest about how AI was involved
- Deployment: verify before you share or ship

Two loops



- Description ↔ Discernment: the working loop
- Delegation ↔ Diligence: the responsibility loop

A context file is a description that doesn't go away

```
# Project: portfolio homepage
## Goal and audience
- Scholarship panel, 2 minutes
## Boundaries
- Never invent achievements
## Check
- One clear next action
```

Goal · inputs ·
boundaries · a check.
Write it once.

Two-minute exercise: make context usable

- 0:00–1:00 Write goal, input, boundary, check.
- 1:00–2:00 Partner: name one missing fact.
- Save as project-context.md, or keep it in chat.

Live demo · same task, better context

- A: portfolio opening, no brief
- B: fresh chat + project-context.md
- Compare: audience · facts · next action

Index files: a map for a body of knowledge

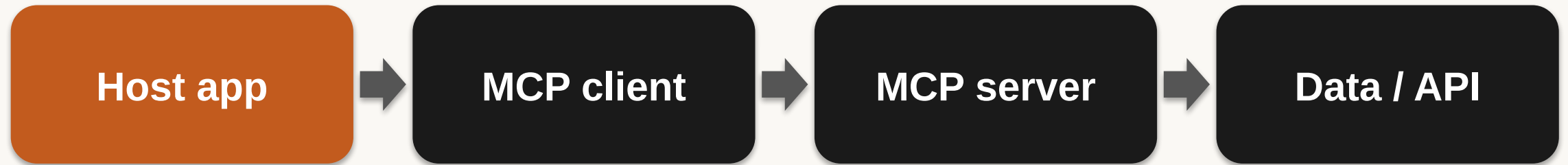
```
# Course knowledge – index
- syllabus.md: schedule, policies
- week-06/lesson.md: context, 4D, MCP
- projects/anim/context.md: my brief
- glossary.md: terms
```

The AI reads the map,
then opens only what it
needs.

API vs. MCP

- Tools turn results into new context, and can change things
- API: one service's menu of requests, each different
- MCP: one shared way for AI apps to use many tools
- MCP servers often call APIs underneath

MCP: how the pieces fit



- Servers expose tools, resources, and prompts.
- A standard connection is not unlimited access.

Control what it can change

- Start read-only, smallest useful scope
- Ask before sending, paying, deleting, publishing
- A document is data, not your boss
- Keep secrets out; stop when permission is unclear

Context is also how a build becomes a product

- Your context file names an audience, inputs, a check
- That's the first draft of a product brief
- Who besides you would use this, and what do they see first?

Learn this your way

- Paste the lesson into your AI, then ask:
- “Explain the 4Ds using my project: ...”
- “Quiz me, one question at a time.”
- “Interview me to write my context file.”
- Then check it: Discernment applies here too

Q&A · what is missing from your context?

- What should your tool know before it starts?
- What should it never do without asking?
- Next: trios share until 6:05.

Sources

- AI Fluency: Framework & Foundations · Dakan, Feller, Anthropic
- Effective context engineering · Anthropic
- AGENTS.md convention · agents.md
- MCP architecture and tools · modelcontextprotocol.io